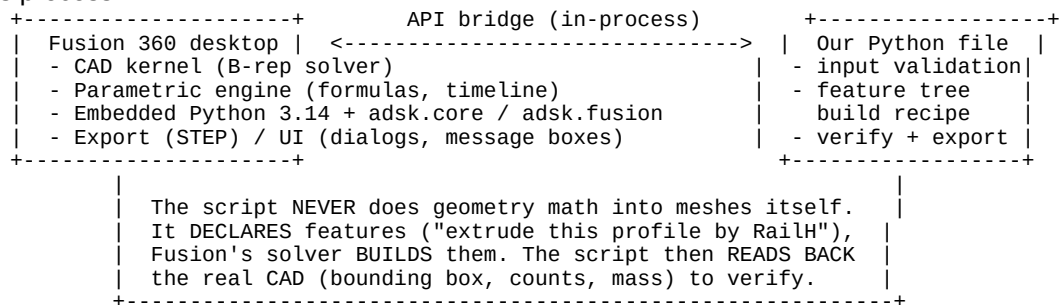


How Our Python Script Drives Fusion 360 — Workflow Explainer

*Companion to FUSION_API_REFERENCE.md (authoritative API detail) and README.md (run instructions). This document explains **how the pieces fit together**: how a .py file talks to a running Fusion 360, what happens on Run, and how our conveyor generator flows from input numbers to STEP + BOM.*

1. The big picture in 60 seconds

Autodesk Fusion 360 ships with its **own embedded Python interpreter** (3.14 in the 2026 release) and injects two API modules into it: `adsk.core` (application, UI, geometry primitives) and `adsk.fusion` (designs, sketches, features, patterns, export). Our script is an ordinary Python file that `import adsk...` — but it only works **inside Fusion**, because those modules exist solely in Fusion's process.



This declare-then-verify loop is the whole workflow: **Python proposes, Fusion disposes, Python audits.**

2. How Fusion finds and runs a script

- Location.** A script is a folder containing a .py file (plus an optional .manifest), placed under Fusion's Scripts folder (%APPDATA%\Autodesk\Autodesk Fusion 360\API\Scripts on Windows), or added ad-hoc via **Utilities** → **Scripts and Add-Ins** → ■.
- Launch.** Pressing **Run** loads the file into Fusion's embedded interpreter and calls `run(context)`. When it returns, the script unloads — nothing of ours stays resident (that persistence is what distinguishes an *add-in*, which also implements `stop(context)` and reacts to UI events; see §9).
- Our guard.** The first lines of `run()` do three things:
 - `app = adsk.core.Application.get()` — handle to the running app,
 - `design = adsk.fusion.Design.cast(app.activeProduct)` — the open parametric model (bails out with a message box if you are in Drawings/Manufacture or have nothing open),
 - a **Parametric-design gate** — UserParameters, timeline patterns and suppression only exist in Parametric mode, not Direct mode.

Outside Fusion (CI, editors), `import adsk` fails — so the file wraps it in `try/except ImportError: adsk = None`, and all pure math (derive/verify/BOM) is importable and unit-testable with system Python. That split — **Fusion-dependent shell, Fusion-free core** — is why tests/ run 17/17 without CAD installed.

3. The object chain (the one path to memorize)

Every Fusion API call hangs off this chain:

```
app = adsk.core.Application.get() # the running Fusion
```

```

ui      = app.userInterface           # dialogs, message boxes
design  = adsk.fusion.Design.cast(app.activeProduct) # open model
root   = design.rootComponent        # top assembly
occ    = root.occurrences.addNewComponent(adsk.core.Matrix3D.create())
comp   = occ.component                # ← we build EVERYTHING here

```

From comp we use five collections, each owning one stage of the build:

Collection	Role in our build
design.userParameters	Named numbers + formulas (ConvLength, RollerCount = floor(...) + 1, ...)
comp.constructionPlanes	Offset work planes (rail level, roller face, guard level)
comp.sketches	2-D profiles, dimensioned to parameters
comp.features.extrudeFeatures	Profiles → 3-D bodies (rails, roller, legs, guards)
comp.features.rectangularPatternFeatures	1 roller → N rollers; 1 leg pair → M stations
design.exportManager	STEP files out

4. Parameters: the parametric backbone

A UserParameter is a **named, unit-aware expression** — "1400 mm", or a formula such as $\text{floor}((\text{ConvLength} - 2 * \text{RollerMargin}) / \text{RollerSpacing}) + 1$. Counts (RollerCount, LegCount) are unitless formulas, so Fusion's own constraint solver recomputes them whenever a base dimension changes — even when the user edits values by hand in *Modify* → *Change Parameters*. Our Python never loops "make N rollers"; it declares **one master roller** and wires the pattern quantity to the formula:

```

qty = adsk.core.ValueInput.createByString("RollerCount") # live formula, not a number
gap = adsk.core.ValueInput.createByString("RollerSpacing") # centre-to-centre pitch
pattern_input = patterns.createInput(bodies, comp.xConstructionAxis,
                                     qty, gap, SpacingPatternDistanceType)

```

Two rules that prevent entire bug classes:

- **createByString vs createByReal**. String inputs ("RailH", "RollerCount") stay live equations; createByReal(5) bakes a dead number in database units. Anything a reviewer should be able to move from the Parameters dialog must arrive via string.
- **Database units are cm**. Point3D.create(x, y, z) takes centimeters and boundingBox returns centimeters, while our spec is millimeters — so sketch placeholders are written in cm and bbox reads are ×10. ValueInput.createByString("1400 mm") converts for you; raw points do not.

Reconfiguration is then just editing expressions in place and calling design.computeAll() **once** — never rebuilding the tree per config (which would duplicate geometry and bloat the timeline ~25 entries per run).

5. Sketches → extrudes → patterns: our build, stage by stage

```

UserParameters (L, W, H, D, P, S, G + formulas)
  ▼
[A] Rails sketch ■■dimensioned: ConvLength × RailW, ConvWidth■■■ extrude RailH
  ■
[B] Master roller sketch ■■dia RollerDia @ (RollerMargin, FrameHeight)■■■
    extrude (ConvWidth - 2·(RailW + clearance))
    ■
    RectangularPattern × RollerCount @ RollerSpacing (Identical compute)

```


Layer	Needs Fusion?	Tested by
Ranges, roller_count_for, leg_count_for, positions, verify, signature, BOM, CSV writer	No	tests/test_conveyor_generator.py (11)
Param create/update, tree build, validator, STEP, batch (mocked adsk)	Mocked	tests/test_fusion_api_mock.py (6)
Real B-rep solve, timeline, export files	Yes — one Mode-1 run	validation_report.txt

9. Script today, add-in delivered (hybrid path)

The file began life as a **script**: runs once, inputBox/messageBox UI, then unloads. It now ships with a persistent companion, conveyor_addin/, built on the AddInSample skeleton with the BulkExportSketchesAsDXF dialog pattern (see FUSION_API_REFERENCE.md §15): typed CommandInputs (mm value fields + guard checkbox), a **live preview** box fed by the pure derive_configuration core on every inputChanged, an **Export STEP + BOM** button with a stay-open status log, and model reuse — find_existing_model reconfigures the existing ParametricConveyor_Assembly instead of rebuilding it. The script remains the default entry; the add-in reuses every function in §§3–8, so nothing here is thrown away.

10. If something goes wrong

- **"No active design"** → open/create a Model design (not Drawing/Manufacture).
- **"Switch to Parametric mode"** → Direct mode has no parameters/timeline.
- **Geometry 10× off** → mm passed where cm expected (or bbox read without ×10).
- **UI param edit does nothing** → that input arrived via createByReal or a sketch dimension lacks its expression — search for the parameter name.
- **Stale numbers** → a read happened before computeAll().
- **adsk import errors in VS Code** → expected: develop pure-Python locally, run CAD in Fusion (or its Edit-bridge VS Code). No stubs are vendored by policy.